LR-Agent / Agent 后端入门

第00章
学习前检查与讲解规范

从最基本的 Python 出发，逐步学会读 Agent 后端源码、完成实验，并形成能被面试追问的个人项目经历。

读者起点

只掌握最基本的 Python；LangChain 从零开始。每次先读一个小例子，运行并解释结果，再进入项目源码。

学完本章应该能够

- 知道先补哪些语法，什么时候才开始看 Agent 内核。
- 理解本项目后端的学习范围，找到当前资料入口。
- 区分“项目已有能力”和“自己亲自完成的工作”。
- 为每章留下实验、源码定位和面试复盘证据。

范围：Python、LangChain 与 Agent 后端。外部调用方只作为接口参与者，不安排前端实现学习。
基线：2026-10-17，d5dd009；LangChain 1.4.2 / LangGraph 1.2.12。

本章目录与阅读方法

0.1 先明确：我们到底学习什么 3

0.2 为什么重新安排入门顺序 4

0.3 文件数量按知识依赖划分 5

0.4 项目现在是否适合学习 6

0.5 沿用飞书讲法，补上学习台阶 7

0.6 先用后端视角认识业务 8

0.7 从学习记录积累简历证据 9

0.8 学到哪一步，回答哪类追问 10

0.9 开始学习前的最后检查 11

每节按“具体问题 → 名词解释 → 小例子 → 输出解释 → 项目对应 → 练习”推进。若代码出现陌生语法，先返回第 1 章相应小节，不要求提前掌握整门 Python。

蓝色标题可通过目录或书签跳转。完整实验由读者自行新建文件，不依赖额外附件。源码节选用于阅读，不能直接当完整脚本运行。

0.1 先明确：我们到底学习什么

假设用户说：“请给这两张图片标注汽车，先让我确认。”本课程要研究的是：后端怎样理解输入、给模型提供工具、执行或委派调用、处理结果与错误，以及限制重复操作。

学习对象	本课程的重点	学习深度
Python	读懂函数、数据、类、校验与异步	先补够用的语法，再读源码
LangChain / Agent	消息、模型、工具循环、运行上下文	核心主线，完整实验
FastAPI / 协议	请求校验、SSE、取消、结果续跑	后端工程重点
推理与外部执行方	输入输出、执行归属与确认状态	理解接口，不学习前端实现

两个“后端”目录，职责不同

LR-Agent/vendor/local-agent：主修目录，负责模型适配、Agent、工具、流式响应和业务约束。虽然名字含 local，它依然是 Python 后端服务。

LR-Agent-backend：认证和用户资料等服务。只补与项目相关的认证、数据库边界，不另开一门全栈开发课。

范围约定

不安排 Electron、React、页面或桌面内部实现学习。文中“外部调用方”表示请求和结果来自哪里，读者不需要打开其前端代码。RAG 不进入本项目改造范围。

0.2 为什么重新安排入门顺序

上一版先讲 Context、业务中间件和状态门控。对只会基础语法的读者，这些内容会同时引入类、继承、类型标注、工具循环和业务规则，容易只能记住术语。



图 0-1 教学顺序示意，不是项目运行时的业务流程。

你不需要先学完所有 Python 高级特性。每出现一个必要语法，就补一个小实验。例如先知道 yield 会逐步产出，再去学习 SSE；先理解类和字段，再看 AgentContext。

每章如何判断“可以往下学了”

能预测一个小例子的输出，运行后解释差异；能定位一个项目符号，说清输入与返回；能回答一个“为什么这样做”。只看懂页面或背出定义还不够。

0.3 文件数量按知识依赖划分

暂定 10 份 PDF，第 0 章负责导航，第 1 至 8 章递进讲技术，第 9 章整理个人改造与面试。第 0、1、2 章当前已交付，其余是待制作计划。

章节	主题	本章要回答的问题
00	学习前检查与讲解规范	怎么学、学到什么程度、怎么留证据?
01	读懂 Agent 所需的 Python 基础	这些源码语法是什么意思?
02	LangChain 模型、消息与链	输入如何变成一次模型响应?
03	工具调用与 Agent 核心循环	谁选择工具，谁负责执行与结束?
04	Middleware 与业务约束	怎样门控、去重和控制执行预算?
05	FastAPI 流式接口与请求续跑	结果怎样回传，怎样取消和防重复?
06	MCP、子代理与上下文记忆	怎样扩展能力，怎样约束边界?
07	模型适配与推理工具接入	语言模型和视觉/推理工具如何分工?
08	后端测试、调试与证据	怎么证明功能正确，怎么定位失败?
09	个人改造与简历面试复盘	哪些工作是我做的，能经得住哪些追问?

表 0-1 教材规划。后续根据内容负担调整篇幅；不为固定数量硬凑章节。

0.4 项目现在是否适合学习

结论是基本适合。项目有真实的工具调用业务、官方 create_agent 工厂、结构化请求、业务约束和测试，能够支撑从小实验进入真实源码。

已有基础	源码/记录中的事实	教学处理
内核入口	Settings.agent_engine 默认 candidate	解释历史配置名，不改 API
框架	LangChain 1.4.2 / LangGraph 1.2.12	按当前环境验证，不照搬课程版本
执行边界	后端工具与外部工具分工	先讲请求/结果，不讲前端实现
业务闭环	已有两图真实推理、确认写盘与回读验收	引用已有记录，另做入门实验
历史说明	部分入口注释曾落后于当前实现	已修正主要入口，旧文档当背景

开始前只做三项检查

1. 确认能打开项目和 study 目录。
2. 按第 1 章指定解释器执行离线代码。
3. 确认自己能解释列表、字典与一个简单函数的输入输出。

不必从安装全部组件开始

入门实验不依赖桌面端、数据库、远程模型或图像推理。正式 Agent 环境已经存在；后续需要某个服务时，再解释它的职责与启动步骤。

正式解释器：D:\Study\LR-Agent\.venvs\langchain-main\Scripts\python.exe。依赖入口：LR-Agent/vendor/local-agent/requirements.txt。

0.5 沿用飞书讲法，补上学习台阶

模仿的是“为什么需要它 → 它是什么 → 怎么用 → 结果说明什么”的节奏。课程目录不是必须复制的结构；项目不存在的机制不能为了完整而写成已有能力。

每节内容	读者要得到什么	具体要求
问题与场景	知道为什么学	用两张图片、一次工具调用等小输入
名词与语法	知道每个新词的含义	中文解释 + 英文/符号 + 一个例子
完整实验	能预测、运行、改输入	全部导入、输入、命令与实际输出
结果与源码	连接实验和真实实现	解释关键行，给真实文件和符号
回顾与面试	能用自己的话复述	练习、参考思路、设计取舍与证据

三种代码标记

完整教学实验：可复制运行，但可能用固定模型或教学数据。
源码节选：展示项目局部，前置变量或导入未必包含。
教学简化示例：为了看懂一种机制，明确说明省略的生产约束。

排版沿用蓝色编号标题、短段落、带行号代码和浅色提示框。先把代码拆短，再考虑字号；不能用一整页密集源码替代讲解。

0.6 先用后端视角认识业务



图 0-2 后端典型任务路径的简化图；省略错误、取消及普通聊天分支。

这时只需建立一个直觉：模型提出“做什么”，确定性代码负责“怎么执行与校验”。外部执行方负责某些实际操作，后端不能把“已经提出调用”当成“已经完成保存”。

现在先不用背的概念

Context、State、Checkpoint、Middleware 会在有了消息和工具循环体验后逐一解释。当前工厂没有图 checkpointer；请求续跑与业务快照也不是同一个机制。先记住“数据保留在哪里、谁重建它”是之后要回答的问题。

0.7 从学习记录积累简历证据

本项目可用于积累 Agent 后端经历，但“仓库里有这个功能”不能直接写成“我从零实现了它”。每章都分别记录理解、复现和亲自改造。

贡献层级	应该保存的证据	适合怎样表达
阅读理解	源码位置、调用链、自己的解释	理解并分析了某条执行链路
亲自复现	命令、输入、输出、失败条件	在本地复现并验证某个行为
亲自改造	本人提交、修改前后行为、测试	针对具体问题实现某项后端改进

可填写的简历结构

面向【具体业务问题】，基于现有项目负责【本人实际完成的模块/改造】，采用【技术与设计】实现【可观察行为】，通过【实验或测试】验证【结果】，并明确【限制】。

这是填写结构，不是现成经历

未完成的改造不写成已完成；已有测试总数不写成本人新增数量。准确率、耗时、成本和并发数字必须有对应实验，不能由一次演示推断。

每章留下一页自己的记录

写清：我原先不懂什么；哪个输入让我理解了；我打开了哪个函数；实际输出是什么；还有哪个失败场景没覆盖。以后面试的细节主要来自这些记录。

0.8 学到哪一步，回答哪类追问

阶段	可以练习的追问	需要的准备
第 1、2 章	消息为什么分角色？链与函数有什么区别？	能运行小例子并解释数据类型
第 3、4 章	为什么用 Agent？怎样限制工具行为？	能追踪工具循环与业务门控
第 5 至 7 章	怎样处理重复结果、取消和外部工具失败？	能说明协议、ID 与实际实现边界
第 8、9 章	你改了什么？如何证明有效？还有什么缺陷？	本人提交、失败案例和验证记录

一次后端改造应当多小

例如，为一个明确的重复结果输入补合同测试，或为一种工具失败补更清楚的反馈。先复现问题，再说明改动位置，最后验证正常与失败分支。比堆叠新组件更容易形成完整的设计与证据。

这些是后续学习任务，本次尚未替你实现，也不代表你已经具备相应贡献。LangChain 的使用、业务约束的设计、异常处理和测试思路都可以成为追问点，前提是能够具体解释。

练习分两种回答

先给 30 秒定义，再用 1 至 2 分钟讲项目中的输入、执行步骤、设计取舍和证据。不要把整段教材当成必须背诵的标准答案。

0.9 开始学习前的最后检查

问题 1：我需要先学桌面开发吗？
参考：不需要。学习后端的请求、工具结果和状态协议即可。

问题 2：读完模型消息一章，是否就能写“独立实现 Agent 系统”？
参考：还不能。还要学习工具循环、约束和接口，并完成属于自己的改造与验收。

问题 3：为什么入门先用固定模型？
参考：把框架行为与模型的不确定回答分开，先验证自己理解的数据流，再进入真实模型调试。

本版做了哪些验证

17 个 Python/LangChain 入门实验使用正式环境执行。它们不联网、不写标注文件；固定输出不能用来证明真实模型能力。旧版的三个进阶实验留作后续章节的机制参考，不再作为第 0 章先修。

项目事实来源：当前源码与 docs/2026-10-17本地主流程验收记录.md。该记录中的真实模型、推理和回读结果属于已有验收，本版没有重新执行它们。
教学方法来源：用户指定飞书资料、《学习PDF编写方法与复用模板.md》及两个参考会话。当前范围与细化计划见 docs/学习准备与章节计划.md。

下一步

打开第 1 章，从解释器和容器数据开始。每次只完成一到两个小节，预测输出、运行、修改一个输入，再用自己的话解释。